AI Engine Layers 1, 2 & 4

Codebase review findings - Layer 1 (Computer Vision), Layer 2 (NLP), and Layer 4 (Rule Engine)

1. The eval harness structurally cannot score the OCR path. Entities and claims are matched by segment_id + char-span, but the pipeline re-segments every image from scratch, so its segment IDs/spans never line up with the hand-labeled gold segmentation. Even a perfect extractor would score ≈0 on 5 of the 6 gold assets. The one metric that doesn't depend on segment IDs — routing — is exactly the one that passes (0.909 F1).

2. The gold labels and the extractor follow different conventions (e.g., gold calls "4.50% APY" a percentage, the extractor emits rate for the same string), so even ID-aligned text assets score 0.

3. Real NLP defects exist underneath: two broken regex/token patterns, marketing headlines misroled as "disclosure" (which silently suppresses their claims), a keyword-count product classifier that misfires on real ads, and a semantic-embedding fallback that has never once fired on any real asset — the embeddings are effectively dead code, not "inaccurate."

Latest gold eval (eval_20260710T114303Z.csv): entities F1 0.000 (all types), claims recall 0.111 / precision 0.077, segments accuracy 0.333 (53 predicted vs 33 gold), routing 0.909 PASS. Silver run (30 assets): segments 0.040 (1,378 predicted vs 201 gold), claims precision 0.010, ECOA recall 0.

1. Eval methodology — why the numbers are near-zero regardless of extraction quality

- Entity scoring uses exact-span mode. eval/run_eval.py:168 calls scorer.entity_metrics(...) with the default match_mode="span" (eval/scorer.py:104), which requires same entity_type + same segment_id + identical char span. The scorer already has a "value" mode built for exactly this situation ("credits extraction that found the right number in a slightly different span", eval/scorer.py:131) — it is never used. That's why amount shows 9 gold / 9 predicted and still 0.0 F1.
- Claim matching requires the same segment_id (eval/scorer.py:236). Predicted IDs come from the pipeline's own OCR segmentation; gold IDs from human segmentation. For carousels it's worse: the pipeline renames IDs to seg_c<card>_<n> (pipeline.py:121) while gold/silver labels use plain seg_<n> — a guaranteed 100% mismatch on every carousel asset, and the silver corpus is mostly carousels.
- Segment accuracy requires exact text equality after whitespace normalization (eval/scorer.py:194), so it conflates three unrelated things: OCR character fidelity, block-merge boundaries, and role classification. A correct role on slightly-different OCR text counts as wrong.
- CV findings pair by (measurement, segment_id, related_segment_id) (eval/scorer.py:321). Because both sides independently number segments seg_1, seg_2, …, unrelated segments collide on the same key — that's why marker_presence shows a perfect MAE of 1.0: gold seg_6 (an asterisk footnote) is being compared against the pipeline's unrelated seg_6.
- Net effect: 4 of the 5 quality bars measure segmentation alignment, not the component they're named after. Until scoring aligns segments by text/bbox overlap first (or entity scoring switches to value mode), you cannot see whether the classifier is actually improving.

2. Gold-label ↔ extractor convention conflicts (these fail even with perfect alignment)

- rate vs percentage: eval/gold_label_format.md:41 and data/gold/sample.json label "4.50% APY" as percentage; the extractor's rate pattern [_NUM, _PCT, {"LOWER": "apy"}] (nlp/entities.py:45) emits rate — and nlp/entities.py:203's own demo asserts rate for that exact sentence. The repo disagrees with its own labeling guide. This alone zeroes the rate/percentage buckets in both match modes (value mode also gates on type).

- APR span conventions: gold labels "25.33%" (just the number) as the apr entity; the extractor pattern [{"LOWER": "apr"}, _ANY(2), _NUM, _PCT] (nlp/entities.py:42) matches "APR of 25.33%" — different span, different raw_text → span-mode miss on every prefix-style APR.

- marker_presence means different things: gold uses it for asterisk-footnote markers (method: "asterisk_footnote_detector", data/gold/881224884306098.json); cv/markers.py only detects sponsorship markers (#ad / sponsored / paid partnership), and the pipeline emits one finding for every segment (51 predicted vs 5 gold), flooding the metric.

- contrast_ratio is labeled but never produced: cv/measure.py:30 implements the WCAG math, but nothing in pipeline.py samples pixel colors — 8 gold findings, 0 predictions, permanently.

- Role conventions: the labeling guide says disclosure = "content required by regulation"; _keyword_role (nlp/normalize.py:41) checks the LEGAL regex before DISCLOSURE, so "APY accurate as of… Member FDIC." → legal, gold says disclosure. Deterministic role misses on mixed segments.

3. Genuine bugs in the NLP layer (layer 2)

- nlp/claims.py:61 — the pricing trigger \b\$\d can never match normal ad copy. \b requires a word↔non-word transition; between a space (or start-of-string) and $ there is none, so "Get $500 today" does not match. Dollar-amount pricing claims are only caught when the entity route happens to fire — on noisy OCR text where entity extraction fails, they're silently dropped.

- nlp/entities.py:54-55 — "60-month term" is unextractable. spaCy's English tokenizer splits 60-month into ["60", "-", "month"]; pattern 1 ([_NUM, unit]) fails because of the intervening - token, and pattern 2 (single-token regex ^\d+-(…)s?$) fails because it's no longer one token. Gold entity ent_10 ("60-month") is a guaranteed false negative. The __main__ demo only tests the spaced form, which hides this.

- Marketing headlines misroled as disclosures kill claim recall. _DISCLOSURE_RE (nlp/normalize.py:21) matches rates? (from|as low as|…) — pure marketing voice. A headline "Rates from 8.99% APR" gets role disclosure, and detect_claims skips disclosure/legal segments entirely (nlp/claims.py:67,114), so the ad's central pricing claim produces no candidate at all. The same misrole also flips the segment onto the "disclosure" side of the proximity measurements.

- nlp/entities.py:180 — assert text[start:end] == raw is a tautology (raw was assigned from the same slice two lines earlier). The "span-integrity invariant" verifies nothing.

- Unpinned spaCy version risk: _ANY = {"OP": "{0,2}"} (nlp/entities.py:24) requires spaCy ≥ 3.5; there is no requirements.txt / pyproject.toml anywhere in the repo, so on an older spaCy the Matcher raises at startup.

4. Routing / product-classifier weaknesses (the "classifier accuracy" problem on real data)

route() picks the product by unweighted keyword counts over the concatenated full text (nlp/routing.py:96-100), which the shakedown report shows misfiring on real ads:

- \bsavings\b in the deposit rule matched a retail "VALUE PACK 100 SAVINGS" ad → routed deposit_account → Reg DD (shakedown asset 1073608245839934).

- BMO business banking brand ads and a US Bank loyalty ad (no deposit terms at all) all routed deposit_account → Reg DD + CFPB — 7 of 11 shakedown assets landed there.

- \breturns?\b → investment (matches "free returns"), \bpolicy\b → insurance (matches "privacy policy"), and "Member FDIC" boilerplate feeds \bfdic\b deposit hits, which can outvote the actual product on bank loan ads.

- The router can never emit ECOA, FDIC, FCRA, or STATE_UDAP. Silver labels route 3 assets to ECOA → recall 0 (silver_run.log); the gap is acknowledged in rule_engine/checks_regb.py:4 ("router never emits ECOA — taxonomy change request #3") and in eval/test_rule_engine.py's NOT_EXERCISABLE list, but it's still open, so checks_fdic328/checks_regb can only run via their universal registration.

- CFPB is added to every credit/deposit ad by design ("over-inclusion is cheap"), but gold labels don't always include it → CFPB precision 0.667. The code's triage philosophy and the eval's precision-bearing F1 bar are in direct policy conflict.

5. Embeddings — not inaccurate, unused

- The semantic fallback has never fired in practice. Every shakedown asset shows fallback: routing=False, sem claims=none. Both fallbacks are gated to fire only when rules produce nothing (nlp/routing.py:105, nlp/claims.py:131), and SEMANTIC_CLAIM_THRESHOLD = 0.6 is high for MiniLM-L6 short-text cosine against synthetic reference sentences (typical related-topic scores are 0.3–0.5). Meanwhile SEMANTIC_ROUTE_THRESHOLD = 0.35 is low — the two thresholds were never calibrated on data; there's no threshold-tuning harness at all.
- Severe inefficiency when it does run: semantic_similarity() re-encodes both texts on every call (nlp/embeddings.py:39-44); _semantic_claims calls it once per claim type, so a rule-less OCR junk segment ("M", "yes", "BMO") costs 12 model encodes; reference sentences are re-embedded for every segment of every asset. Reference vectors should be computed once and cached; segments batched.
- No failure isolation: the fallback imports and downloads the HF model lazily inside detect_claims; offline or on a cold cache, the entire pipeline crashes rather than degrading to rules-only.
- _semantic_route embeds the entire concatenated asset text — MiniLM truncates at 256 word-pieces, so on long assets the score reflects an arbitrary prefix.

6. CV layer (layer 1)

- Blank/unreadable image → unhandled crash: run_ocr_local returns [] on bad input by design, but then segments == [] and AssetRepresentation requires min_length=1 segments (schema.py:582), so analyze_asset raises a ValidationError instead of degrading gracefully.
- Over-fragmentation vs label granularity: the block-merge (nlp/normalize.py:93-195) produces ~46 segments/asset on the silver corpus where labels have ~7 (one per card). Single-character OCR junk ("M", "V", "_") becomes standalone unknown segments that pollute claims, routing full-text, and the marker findings.
- Latency: 9–135 s per asset on free-tier CPU OCR (shakedown) — plus the uncached embedding calls above.
- contrast_ratio implemented but never wired (see §2).

7. Rule engine (layer 4) — the strongest layer, two caveats

The registry/check design, mutation harness, retry orchestrator, and determinism guard are genuinely well-built. Issues:

- Garbage-in-pass-out: every asset-gated check keys on extracted entities/product type. With entity recall ≈0 on image ads (7 of 11 shakedown assets extracted zero entities), checks like REGZ-STR-001 report "not triggered" and the engine "passed r1" on everything. The engine's own ALL-GRD-007 flagged this honestly (two "EXTRACTION GAP" anomalies in the shakedown report). Layer 4's guarantees are currently bounded by layer 2's recall.
- Stale doc: checks_fdic328.py:5 claims "schema.Framework has no FDIC value yet" — schema.py added FDIC in the 2026-07-10 bump. Also rule_engine/version.py caches the ruleset hash keyed only by check IDs, so an in-process source edit wouldn't change the hash the docstring promises it would (minor).
- Note: your described architecture has layer 4 verifying the LLM against chunks of the framework texts — that does not exist here. The engine verifies against compiled-in lexicons (deliberately, per its determinism invariant), and adcoherent_regulatory_sources.json is referenced by no code at all. If chunk-grounded verification is the intent, it's an unbuilt component, not a bug.

8. Repo/infra

- .github/workflows/ci.yml is a Node/pnpm pipeline (pnpm lint/typecheck/eval:rules/build) in a Python repo with no package.json — CI cannot pass a single step. .eslintrc.json/.prettierrc are likewise orphaned.
- No dependency manifest (no requirements.txt/pyproject/lock) despite version-sensitive deps (spaCy ≥3.5 matcher syntax, PaddleOCR 3.x API).
- README links point at file:///home/shadowxlord/... (the dev's machine).

What I'd fix first (recommendation only — nothing edited)

1. Fix the measurement before the model: align gold↔pred segments by text/bbox overlap and remap IDs before scoring; switch entity scoring to the existing value mode for OCR assets. Only then will the other fixes become visible in the numbers.

2. Reconcile the entity-type and span conventions (rate/percentage/apr) between gold_label_format.md, the labels, and the Matcher patterns — one of them has to be the source of truth.

3. Patch the two hard bugs: \b\$\d in claims.py, and the hyphenated time-period pattern in entities.py.

4. Stop _DISCLOSURE_RE from swallowing "rates from/as low as" headlines (or stop _SKIP_ROLES from suppressing claims on them).

5. Rework routing from raw keyword counts to weighted/role-aware signals, and add the missing ECOA/FDIC routing rules the rule engine is already waiting for.

6. Make the embedding fallback real: calibrate both thresholds on labeled data, precompute reference embeddings, add a junk/length gate, and wrap it in a try/except so it degrades instead of crashing.

Happy to turn any section of this into a concrete fix plan when you're ready.